

1 SQL TRANSACTION

What is a Transaction?

A **transaction** is a sequence of one or more SQL operations that are treated as **one logical unit of work**.

👉 Either **all operations succeed**, or **none of them do**.

Example idea:

Transfer money from Account A to Account B

- Debit A
- Credit B

If one fails, both must be undone.

Why Transactions Matter

Transactions protect your data from:

- System crashes
- Power failures
- Partial updates
- Logic errors

They guarantee **data integrity**.

Importance

- Prevents **partial updates**
- Protects **data integrity**
- Essential for **financial & critical systems**

ACID Properties (Very Important)

Every transaction follows **ACID**:

Property	Meaning
Atomicity	All operations succeed or all fail
Consistency	Database moves from one valid state to another
Isolation	Concurrent transactions don't interfere
Durability	Committed data is permanent

Transaction Commands in SQL

Command	Purpose
BEGIN TRANSACTION / START TRANSACTION	Start a transaction
COMMIT	Save changes permanently
ROLLBACK	Undo changes
SAVEPOINT	Mark a rollback point

Advantages of Transactions

- ✓ Ensures **data consistency**
- ✓ Avoids **data corruption**
- ✓ Supports **rollback** if error occurs
- ✓ Controls **concurrent access**
- ✓ Reliable for banking, billing, orders

Disadvantages of Transactions

- ✗ Can **reduce performance**
- ✗ Locks tables/rows (may cause delays)
- ✗ Long transactions can cause **deadlocks**
- ✗ Requires careful error handling

Example

sql

```
BEGIN TRANSACTION;  
UPDATE orders SET status = 'Paid' WHERE order_id = 10;  
UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 5;  
COMMIT;
```

- ✓ Both updates succeed → money transferred
- ✗ If any update fails → use ROLLBACK

Where Transactions Are Used

- Banking systems
- E-commerce orders
- Inventory management

- Any multi-step database operation

Transaction Types

Type	Control	Usage
Implicit	Automatic	Simple queries
Explicit	Manual	Critical operations
Autocommit	Automatic per statement	Default mode
Distributed	Multi-DB	Enterprise systems
Nested	SAVEPOINT based	Partial rollback

2 STORED PROCEDURE

Why we use Stored Procedures

We use stored procedures to **store SQL logic inside the database** and execute it whenever needed.

Importance

- Improves **code reusability**
- Enhances **security**
- Increases **performance**

Advantages of Stored Procedures

- ✓ Reusable code
- ✓ Faster execution (precompiled)
- ✓ Reduces network traffic
- ✓ Improves security (no direct table access)
- ✓ Easier maintenance

Disadvantages of Stored Procedures

- ✗ Harder to debug
- ✗ Database-dependent (not portable)
- ✗ Requires skilled developers
- ✗ Can increase DB server load

Example

```
sql

CREATE PROCEDURE GetStudent
    @Id INT
AS
BEGIN
    SELECT * FROM Students WHERE StudentID = @Id;
END;
```

Stored Procedure Types

Type	Purpose
Simple	Fixed logic
Parameterized	Dynamic input
OUT / INOUT	Return values
Transaction-based	Data safety
Dynamic SQL	Runtime queries
Recursive	Looping logic
System	DB maintenance

3 COMBINED IMPORTANCE
(Transaction + Stored Procedure)

Stored procedures often **use transactions internally** to ensure safe execution.

Example Use Case

- ✓ Bank transfer
- ✓ Online shopping checkout
- ✓ Salary processing

Summary Table (Easy to Remember)

Feature	Transaction	Stored Procedure
Purpose	Data safety	Code reusability
Rollback	Yes	No (but can use transaction)
Performance	Slower	Faster
Security	Medium	High

★ **Exam Tip**

Transaction = Safety

Stored Procedure = Performance + Security